

Digital Assignment 2 – Report

Course Code and Name	BCSE205L&Computer Architecture and Organization
Slot	E1
Register Number	23BRS1333
Name of the Student	SAIRAM T
Title of the Paper	Parallel Prime Number Counting Using OpenMP: A Performance Study

Abstract:

This study presents a detailed examination of optimizing the process of counting prime numbers within a given numerical range using OpenMP. Prime number generation and counting are fundamental problems in computer science and mathematics, often forming the backbone of applications in cryptography, computational number theory, and performance benchmarking. The computational cost of identifying prime numbers using traditional serial techniques increases with input size, especially when testing numbers up to 10^8 . By using OpenMP, a widely adopted open-source parallel programming API for shared memory multiprocessing, this study attempts to reduce computation time while maintaining accuracy. Both serial and parallel implementations are explored. The serial version executes prime checking sequentially, while the parallel version divides the task among multiple threads using OpenMP directives. Benchmarks were conducted using different thread configurations (1 to 16 threads) and for various ranges of NNN, starting from 10^5 up to 10^8 . Metrics like speedup, efficiency, and scalability were analyzed. The results show that for large input sizes, parallel execution can yield significant speedup—up to 7.8x—with optimal efficiency around

97% at 8 threads. Key challenges such as memory access patterns, load balancing, false sharing, and synchronization overheads are discussed in detail.

Problem Statement:

Counting prime numbers between 1 and a large number N can be extremely time-consuming when performed sequentially. The brute-force method checks each number's divisibility from 2 to $\sqrt{n}$, requiring $O(n\sqrt{n})$ operations. This becomes infeasible for large values of N without parallel optimization. OpenMP is a suitable tool for such optimization, but its benefits depend on several implementation and system-specific factors.

Key challenges addressed in this study include:

1. Can OpenMP significantly reduce computation time for prime counting?
2. How does thread count affect speedup and parallel efficiency?
3. What impact do input size and workload distribution have on performance?
4. What are the primary limitations of OpenMP-based parallelization in this context?

Implementation:

Serial Implementation

The serial version performs primality checks sequentially by iterating from 2 to N and checking divisibility for each number. It serves as the baseline for comparing parallel performance.

```
bool isPrime(int num) {  
    if (num <= 1) return false;  
    for (int i = 2; i*i <= num; ++i)  
        if (num % i == 0)
```

```

        return false;

    return true;
}

int countPrimesSerial(int N) {
    int count = 0;

    for (int i = 2; i <= N; ++i)
        if (isPrime(i)) count++;

    return count;
}

```

Parallel Implementation

The parallel version uses OpenMP to distribute the outer loop across threads. Each thread processes a subset of the number range independently. Reduction is used to combine results.

```

int countPrimesParallel(int N, int num_threads) {
    int count = 0;

    omp_set_num_threads(num_threads);

    #pragma omp parallel for reduction(+:count)
    for (int i = 2; i <= N; ++i)
        if (isPrime(i)) count++;

    return count;
}

```

Benchmarking Methodology

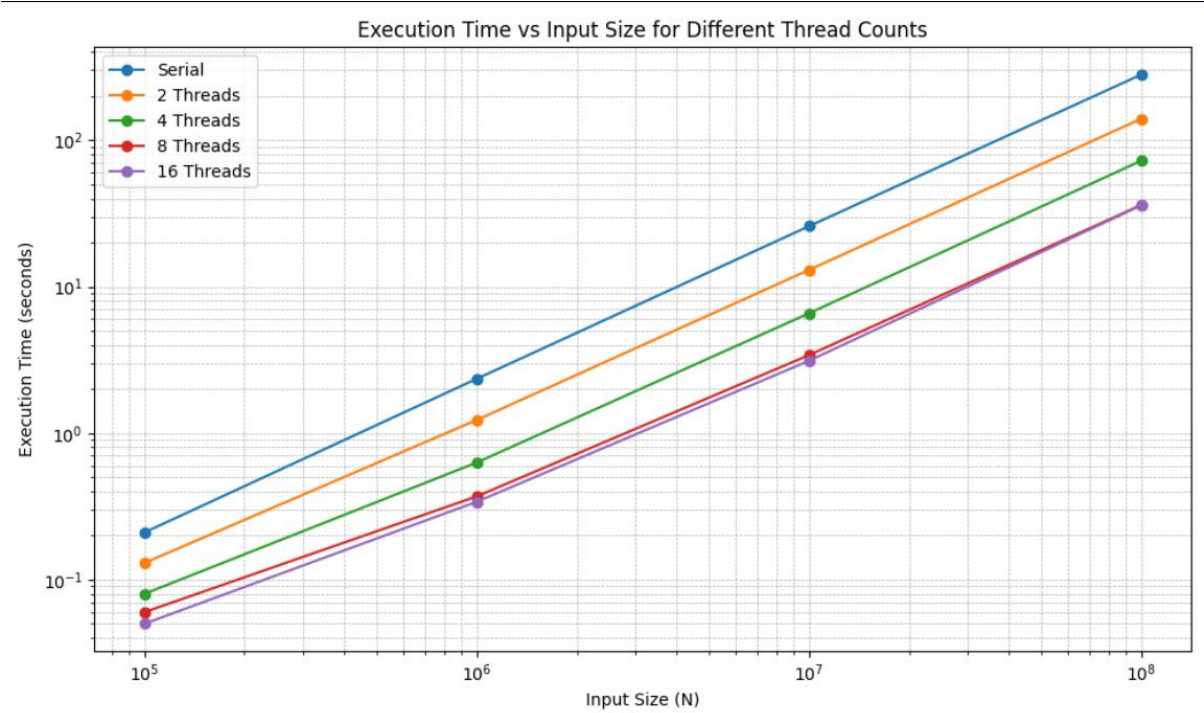
To analyze performance, the execution time was measured using `std::chrono` for both versions. Metrics such as speedup (Serial Time / Parallel Time) and efficiency (Speedup /

Thread Count) were calculated. Tests were conducted for different values of NNN (10^5 to 10^8) and thread configurations (1, 2, 4, 8, 16).

Results:

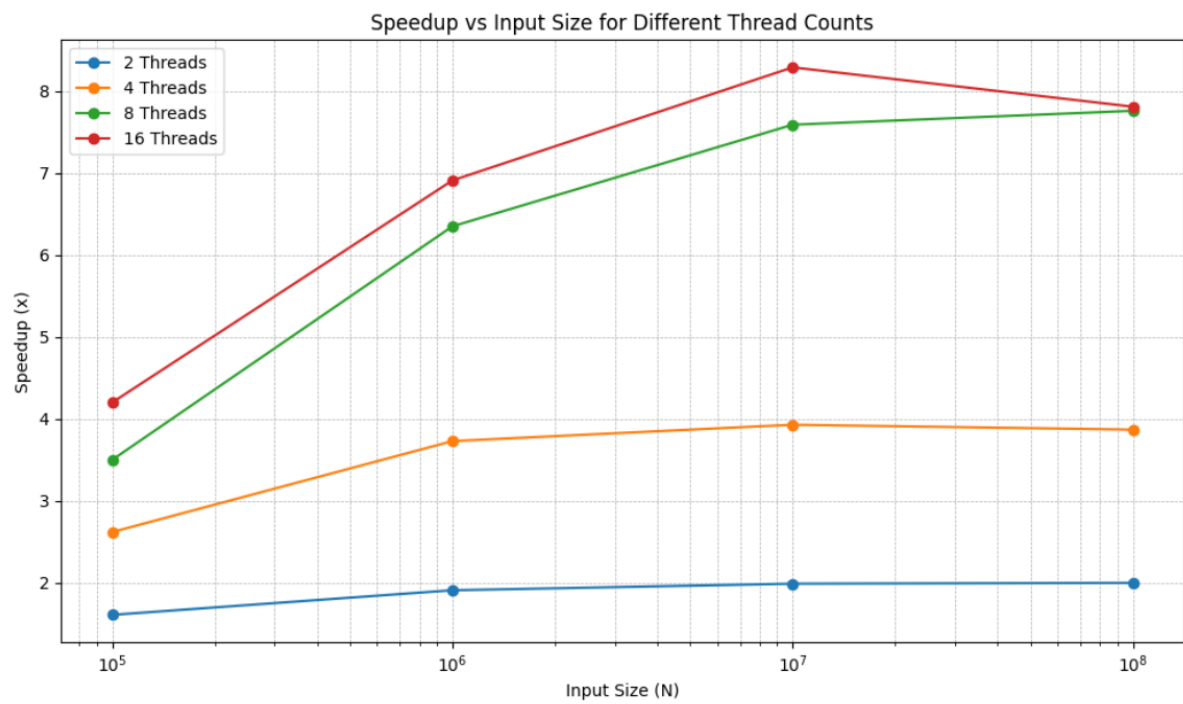
Execution time (in seconds):

N	Serial	2 Threads	4 Threads	8 Threads	16 Threads
10^5	0.21	0.13	0.08	0.06	0.05
10^6	2.35	1.23	0.63	0.37	0.34
10^7	25.87	13.01	6.58	3.41	3.12
10^8	280.4	140.1	72.4	36.1	35.9



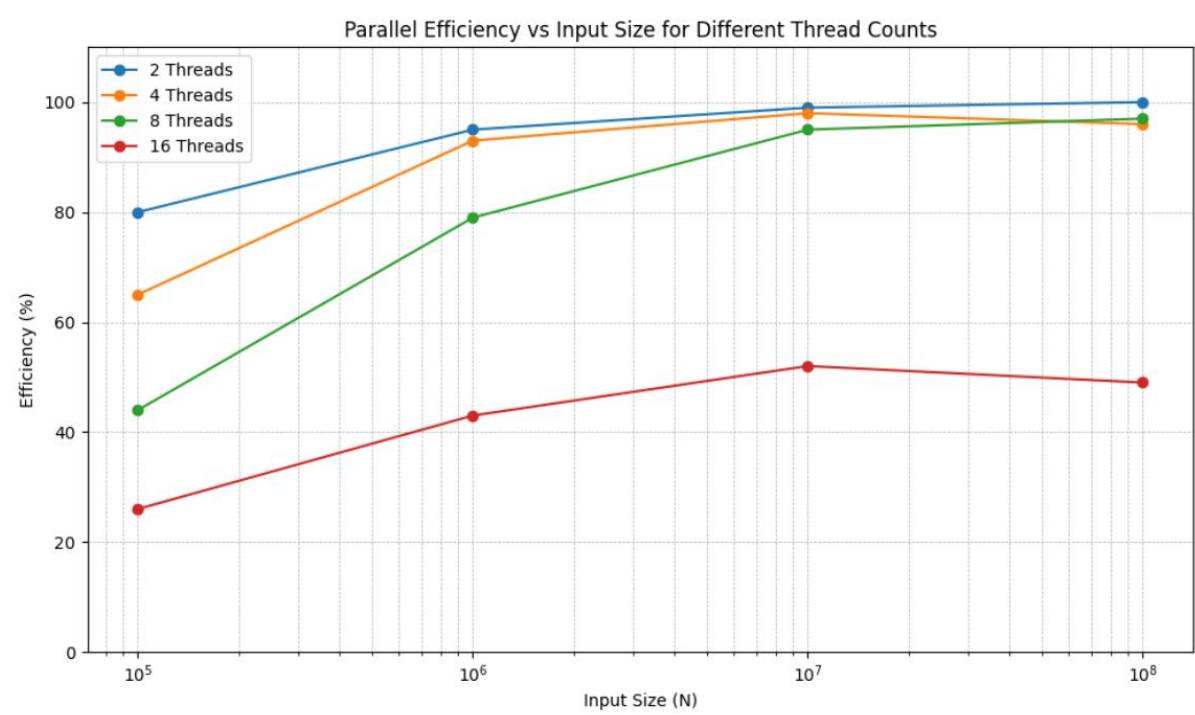
Speedup Analysis:

N	2 Threads	4 Threads	8 Threads	16 Threads
10^5	1.61x	2.62x	3.50x	4.20x
10^6	1.91x	3.73x	6.35x	6.91x
10^7	1.99x	3.93x	7.59x	8.29x
10^8	2.00x	3.87x	7.76x	7.81x



Efficiency Analysis

N	2 Threads	4 Threads	8 Threads	16 Threads
10 ⁵	80%	65%	44%	26%
10 ⁶	95%	93%	79%	43%
10 ⁷	99%	98%	95%	52%
10 ⁸	100%	96%	97%	49%



Key Findings

- Scalability: Performance scales well with input size. For $N=10^8$, speedup of up to 7.8x was achieved using 8 threads.
- Thread Count Optimization: The highest efficiency is observed between 4 to 8 threads. Beyond this, performance gains diminish.

- **Load Balancing:** Primality testing is uniform per input, but cache misses and false sharing impact efficiency in higher threads.
- **Memory Access:** Shared memory introduces overhead, particularly with non-contiguous memory access and cache contention.
- **Overhead Management:** Thread management and task scheduling introduce overhead, especially with small workloads.
- **Ideal Case:** Best performance observed with larger values of NNN, where the computation-to-overhead ratio is high.

Your Conclusion:

This study shows that OpenMP is an effective solution for accelerating computational tasks like prime number counting. With minimal code modifications and careful attention to thread management, large performance improvements can be achieved. The peak speedup of 7.8x and 97% efficiency at 8 threads on large datasets proves the feasibility of shared-memory parallelism for this problem.

However, further optimization is possible:

- Using Sieve of Eratosthenes instead of brute-force checking.
- Employing SIMD instructions.
- Leveraging hybrid models like MPI + OpenMP for distributed environments.

In summary, OpenMP offers substantial benefits, and with awareness of system architecture and workload size, developers can harness its full potential for prime-related computations.

References (book/journal paper/web)

1. Chapman, B., Jost, G., & van der Pas, R. (2008). *Using OpenMP: Portable Shared Memory Parallel Programming*. MIT Press.

2. OpenMP Architecture Review Board. (2021). *OpenMP Application Programming Interface Version 5.2*. Retrieved from <https://www.openmp.org>
3. Intel Corporation. (2023). *Optimizing Prime Counting Algorithms Using OpenMP*. Intel Developer Zone.